

UT 1

Desarrollo de software y ciclo de vida

Entornos de Desarrollo · Formación Profesional

- Del código fuente a una aplicación real
- Cómo trabajan los equipos actuales
- DevOps, seguridad, dependencias e IA

Idea clave: desarrollar software no es solo programar; es transformar necesidades en soluciones mantenibles y seguras.



Qué vamos a aprender

1.1 Qué es un programa informático

1.2 Código fuente, objeto y ejecutable

1.3 Tipos de lenguajes y paradigmas

1.4 Lenguajes más usados en la industria

1.5 Fases del desarrollo de una aplicación

1.6 Herramientas para obtener y ejecutar software

1.7 Metodologías ágiles

1.A-E DevOps, SSDLC, dependencias e IA

Objetivo de clase: entender el “viaje” completo de una aplicación desde una idea inicial hasta que se mantiene en producción.

La idea grande: del problema al producto



- Un programa resuelve una necesidad concreta.
- Una aplicación combina programa, datos, interfaz, configuración y entorno.
- El ciclo de vida organiza el trabajo para reducir errores y mejorar el mantenimiento.

Ejemplo guía: HelpDesk Lite, una miniaplicación para registrar incidencias de usuarios.

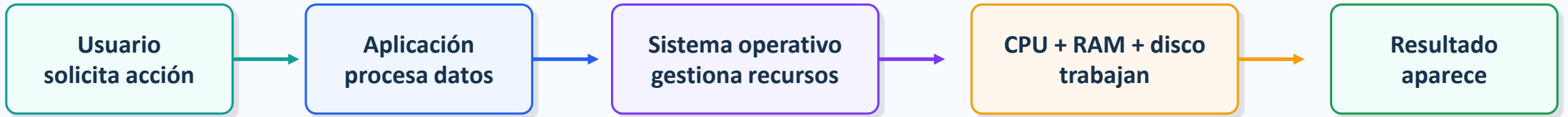
1.1 · ¿Qué es un programa informático?



- Programa: instrucciones guardadas para resolver una tarea.
- Aplicación: solución completa que usa programas, datos y configuración.
- Proceso: programa en ejecución, usando CPU, memoria y recursos del sistema.

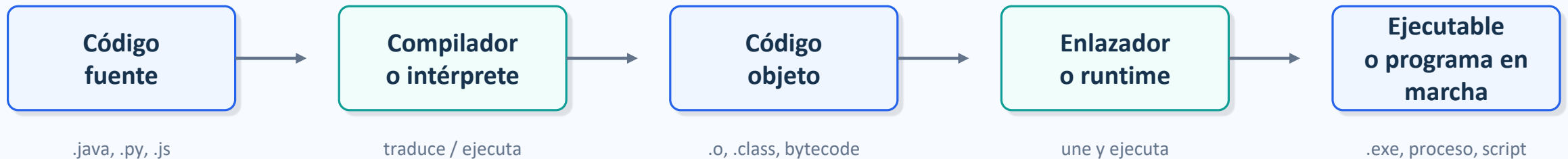
Idea para alumnos: “Chrome.exe” en disco es programa; Chrome abierto en el Administrador de tareas es proceso.

Programa, sistema operativo y hardware



- El sistema operativo es el intermediario entre el programa y el hardware.
- La RAM guarda datos temporales; el disco conserva archivos; la CPU ejecuta instrucciones.
- El mismo programa puede comportarse distinto según sistema, permisos y configuración.

1.2 · Fuente, objeto y ejecutable



No todos los lenguajes siguen exactamente el mismo camino: algunos generan ejecutables nativos, otros bytecode, otros se interpretan o combinan JIT.

- Fuente: legible para programadores.
- Objeto/intermedio: resultado parcial de traducción.
- Ejecutable/runtime: lo que realmente se pone en marcha.

Herramientas que aparecen en el camino

Editor / IDE

VS Code, IntelliJ, Eclipse

Compilador

javac, gcc, rustc

Intérprete / runtime

Python, Node.js, JVM

Gestor de paquetes

pip, npm, Maven, Gradle

Control de versiones

Git, GitHub, GitLab

Build / CI

tests, empaquetado, análisis

Desarrollar es combinar herramientas: escribir, traducir, probar, guardar versiones, empaquetar y desplegar.

Virtualización: tres ideas que conviene separar

1

VM de sistema

Simula un ordenador completo: SO invitado, CPU, RAM, disco virtual.

VirtualBox
VMware
Hyper-V

2

Contenedor

Aísla aplicación y dependencias usando el kernel del sistema anfitrión.

Docker
Podman
Kubernetes

3

VM de lenguaje

Ejecuta bytecode o código intermedio: JVM, CLR, runtimes.

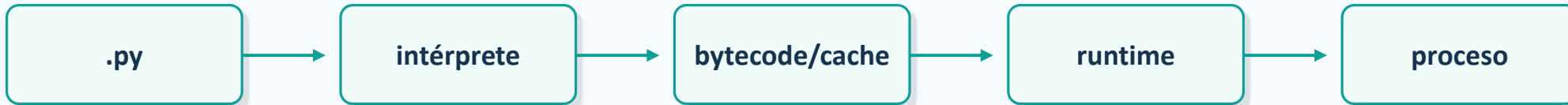
JVM
.NET CLR
V8

Ejemplos: Java y Python no funcionan igual

Java



Python



Conclusión: “compilado” e “interpretado” son simplificaciones. Los runtimes modernos suelen mezclar varias técnicas.

1.3 · Tipos de lenguajes de programación

Por nivel y uso

- Bajo nivel: ensamblador, muy cercano al hardware.
- Alto nivel: Java, Python, JavaScript, C#, PHP.
- Propósito general: sirven para muchos tipos de aplicaciones.
- Dominio específico: SQL, HTML/CSS, Bash, MATLAB.

Por forma de ejecución

- Compilados: generan código nativo o intermedio.
- Interpretados: el runtime ejecuta instrucciones.
- Híbridos/JIT: traducen y optimizan durante la ejecución.
- Declarativos: expresan qué se quiere, no tanto cómo hacerlo.

Evita respuestas absolutas: muchos lenguajes actuales combinan varios modelos.

Paradigmas: distintas formas de pensar el código

Imperativo

pasos y órdenes

Orientado a objetos

clases, objetos, encapsulación

Funcional

funciones, inmutabilidad

Declarativo

describir el resultado

Eventos

reaccionar a acciones

Concurrente

varias tareas a la vez

Un mismo lenguaje puede ser multiparadigma: JavaScript, Python, Java, C#...

1.4 · Lenguajes muy presentes en la industria

Java

backend, Android, empresa

Python

scripts, datos, IA, automatización

JavaScript/TS

web, frontend, backend Node.js

C/C++

sistemas, rendimiento, bajo nivel

SQL

bases de datos y consultas

Rust/C#

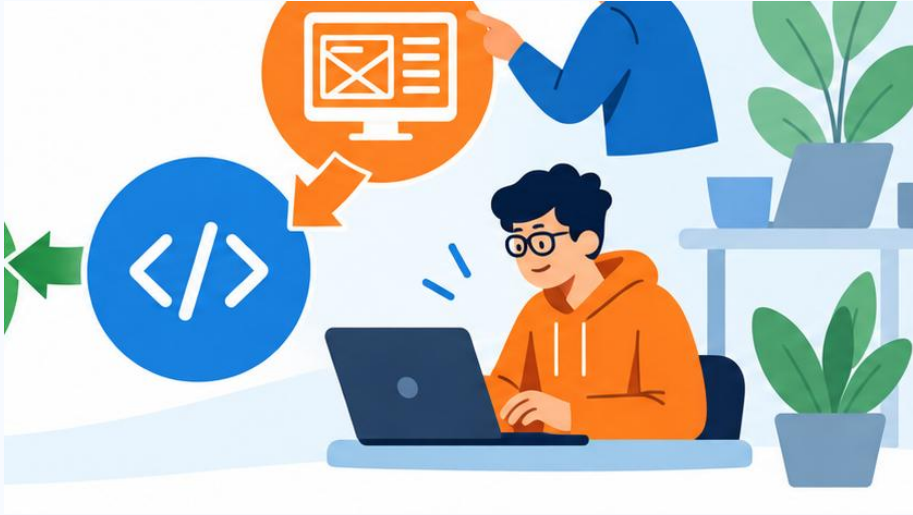
seguridad, rendimiento,
escritorio/servicios

Elegir lenguaje no es “cuál es mejor”, sino cuál encaja con problema, equipo, rendimiento, mantenimiento y ecosistema.

Cómo elegir un lenguaje de forma razonable



1.5 · Fases del desarrollo de una aplicación



- Análisis: entender el problema y los requisitos.
- Diseño: decidir estructura, datos e interfaz.
- Codificación: implementar con calidad.
- Pruebas: comprobar y detectar defectos.
- Documentación, explotación y mantenimiento: usar, mejorar y corregir.

La fase de mantenimiento suele ser la más larga: el software cambia porque cambian los usuarios, errores, leyes, datos y tecnología.

Análisis: convertir necesidades en requisitos

Caso: HelpDesk Lite

- Registrar incidencias con usuario, prioridad y estado.
- Consultar incidencias propias y del equipo.
- Filtrar por estado y fecha.
- Evitar que un usuario vea datos ajenos.

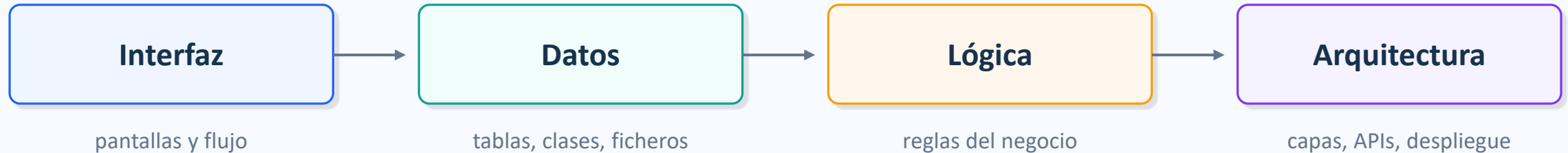
Tipos de requisitos

**Funcionales
qué hace**

**No funcionales
calidad**

Ejemplo no funcional: la aplicación debe responder en menos de 2 segundos para consultas habituales.

Diseño: antes de programar, pensar la solución



- El diseño reduce improvisación y facilita repartir trabajo.
- Permite detectar problemas antes de que sean caros.
- Puede representarse con esquemas, diagramas UML, prototipos o pseudocódigo.

Codificación: escribir código mantenible



Legible

nombres claros, formato coherente



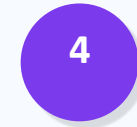
Pequeño

funciones sencillas y responsabilidad única



Probable

validaciones y manejo de errores



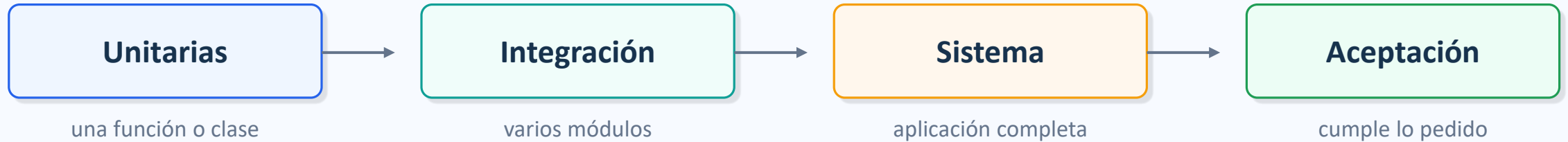
Revisable

commits pequeños y trazables

Programar no es “que funcione una vez”: es dejar una solución que otra persona pueda entender, probar, mejorar y desplegar.

- Usa control de versiones desde el primer día.
- Evita copiar código sin comprenderlo.
- Añade validación de entradas y tratamiento de errores.

Pruebas: no basta con “a mí me funciona”



Criterio de aceptación: condición concreta que permite comprobar si una historia de usuario está terminada.

- Caso normal: lo habitual.
- Caso límite: valor extremo.
- Caso de error: entrada incorrecta.
- Regresión: algo que antes funcionaba no debe romperse.

Documentación, explotación y mantenimiento

Documentar

qué hace, cómo instalar,
cómo usar, decisiones
tomadas

Explotar

poner en funcionamiento
para usuarios reales

Mantener

corregir fallos, adaptar,
mejorar y actualizar

La documentación útil no es “relleno”: permite continuar el trabajo cuando cambian personas, versiones o requisitos.

Modelos de ciclo de vida

Cascada

fases secuenciales; útil si todo está muy definido

Iterativo

versiones sucesivas; aprendizaje progresivo

Ágil

entregas cortas, feedback y adaptación

DevOps

desarrollo + operaciones + automatización continua

No hay un único modelo perfecto: se elige según incertidumbre, equipo, criticidad, plazos y riesgo.

1.7 · Metodologías ágiles



- Entregas frecuentes y pequeñas.
- Feedback temprano de usuarios o cliente.
- Adaptación cuando cambia el contexto.
- Trabajo visible y colaborativo.
- Mejora continua del equipo.

Ágil no significa caos ni ausencia de documentación; significa inspeccionar, adaptar y entregar valor de forma incremental.

Scrum: roles, eventos y artefactos

Roles

Product Owner · Scrum Master · Equipo de desarrollo

Eventos

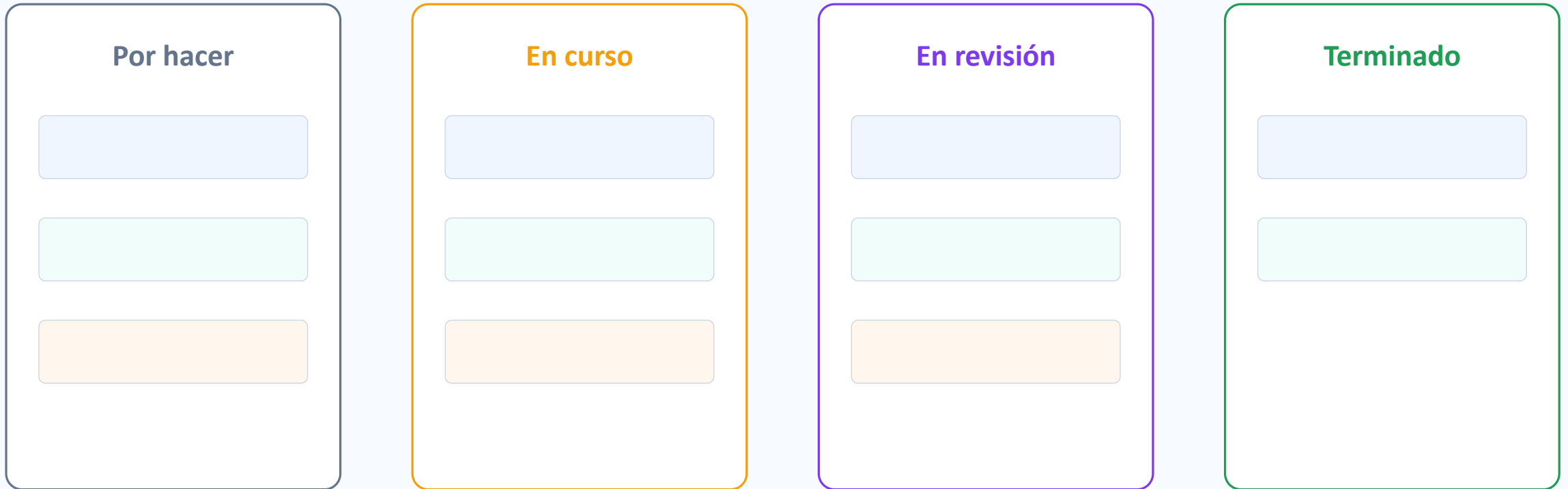
Sprint · Daily · Review · Retrospective

Artefactos

Product backlog · Sprint backlog · Incremento

Sprint: periodo corto, normalmente 1-4 semanas, donde el equipo intenta entregar un incremento útil.

Kanban e issue tracking



- Issue: tarea, error, mejora o investigación que se registra y se sigue.
- WIP: límite de trabajo en curso para no abrir demasiadas tareas a la vez.
- Métrica útil: tiempo de ciclo, desde que se empieza hasta que se termina.

Historias de usuario y criterios de aceptación

Historia de usuario

Como [tipo de usuario], quiero [necesidad], para [beneficio].

Ejemplo

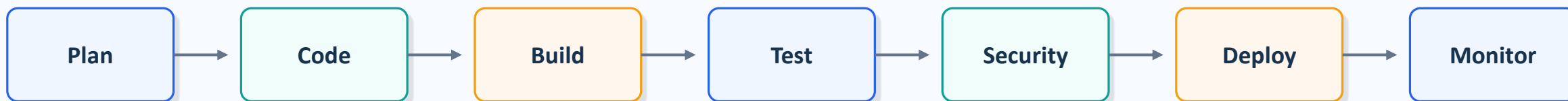
Como técnico de soporte, quiero filtrar incidencias por prioridad para atender primero las más urgentes.

Criterios de aceptación

- Dado un listado con incidencias...
- Cuando selecciono “prioridad alta”...
- Entonces solo se muestran las incidencias altas.

Sin criterios de aceptación, es difícil saber cuándo una tarea está realmente terminada.

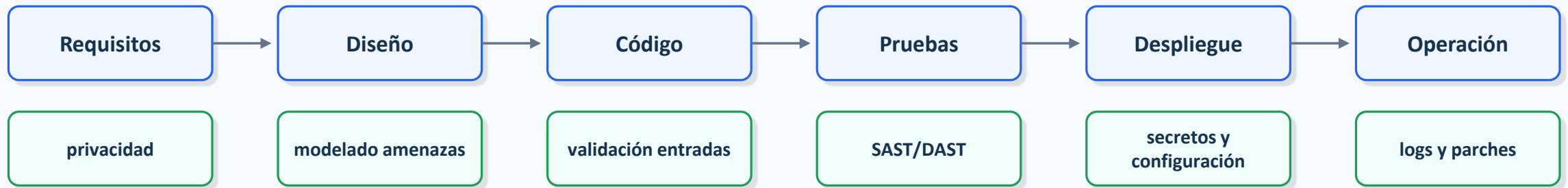
1.A · DevOps: automatizar el flujo de desarrollo



- CI: integración continua; cada cambio se construye y se prueba pronto.
- CD: entrega/despliegue continuo; publicar versiones con menos trabajo manual.
- Pipeline: receta automática con pasos repetibles y trazables.

Objetivo: menos “en mi equipo funciona” y más evidencias automáticas antes de llegar a producción.

1.C · SSDLC: seguridad integrada desde el principio



SSDLC significa no dejar la seguridad para el final. Cada fase debe producir evidencias: requisitos, diseño, tests, revisiones y configuración segura.

- Confidencialidad: que no acceda quien no debe.
- Integridad: que los datos no se alteren sin control.
- Disponibilidad: que el servicio funcione cuando se necesita.

1.D · Dependencias y cadena de suministro



- Una dependencia directa la eliges tú; una transitiva llega a través de otra.
- Riesgos: vulnerabilidades, paquetes maliciosos, typosquatting, versiones abandonadas.
- Controles: versiones fijadas, actualización controlada, SCA, SBOM y revisión de licencias.

1.E · Asistentes de IA para desarrollo



- La IA puede proponer código, tests, documentación y explicaciones.
- El responsable final sigues siendo tú.
- No compartas contraseñas, datos personales, claves API ni código sensible sin permiso.
- Todo código generado debe revisarse, probarse y trazarse.

Flujo recomendado: pedir poco, entender, probar, revisar seguridad, documentar la decisión y mantener solo lo que puedas explicar.

Cómo encajan todas las piezas



Ágil

prioriza y adapta

Git/issues

traza cambios

SSDLC

añade seguridad

Supply chain

controla dependencias

IA

ayuda, pero se valida

Producto profesional = valor para el usuario + calidad técnica + seguridad + trazabilidad + capacidad de mantenimiento.

Proyecto de aula: HelpDesk Lite

1 Problema, usuarios y alcance.

3 Lenguaje/plataforma y modelo de ejecución.

5 Tablero con issues y estados.

7 5 controles SSDLC colocados en fases.

2 5 historias con criterios de aceptación.

4 Diagrama de fases del ciclo de vida.

6 Pipeline con controles automáticos.

8 Política de dependencias e IA.

Entrega recomendada: documento breve + tablero + repositorio Git + breve defensa oral de decisiones.

Repaso final: 10 preguntas rápidas

1. ¿Diferencias programa, aplicación y proceso?
2. ¿Qué cambia entre fuente, objeto y ejecutable?
3. ¿Qué es una JVM y en qué se diferencia de una VM de sistema?
4. ¿Por qué un lenguaje puede ser multiparadigma?
5. ¿Qué fase evita requisitos ambiguos?
6. ¿Qué prueba comprueba que se cumple lo pedido?
7. ¿Qué aporta Kanban al equipo?
8. ¿Qué automatiza un pipeline DevOps?
9. ¿Qué controla SCA en dependencias?
10. ¿Qué debes hacer antes de integrar código generado por IA?

Para estudiar: explica cada concepto con un ejemplo propio. Si solo memorizas definiciones, te faltará criterio técnico.